1. 8 PUZZLE PROBLEM

Aim

To solve the **8-Puzzle Problem** by comparing the given initial state with the goal state and determining the number of misplaced tiles using Python.

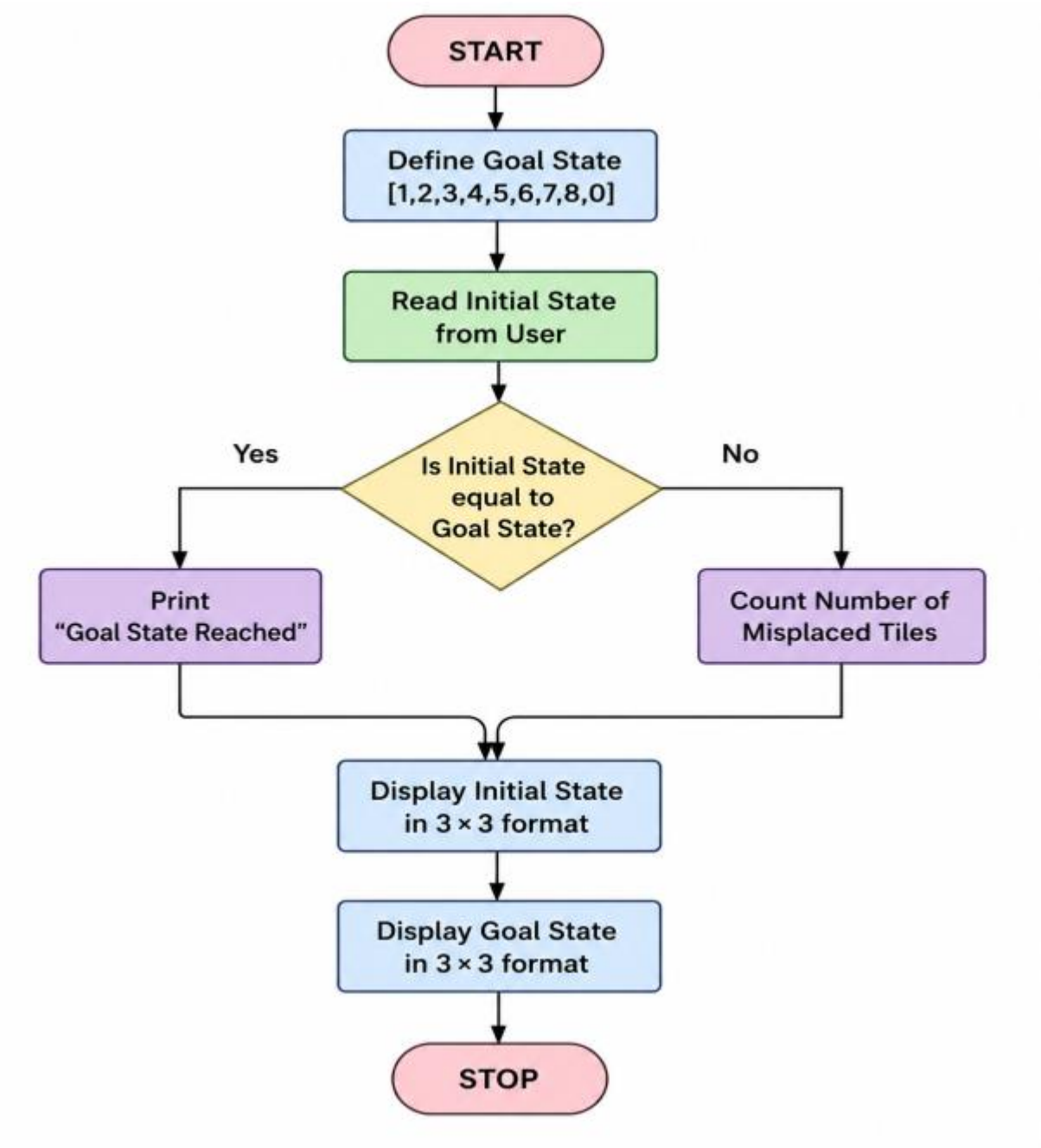
Objective

- To understand the concept of the 8-puzzle problem.
- To represent the puzzle using a 3 × 3 matrix.
- To compare the initial state with the goal state.
- To calculate the number of misplaced tiles.
- To display both the initial and goal states.

Algorithm

1. Start the program.
2. Define the goal state as **[1,2,3,4,5,6,7,8,0]**.
3. Read the initial state from the user.
4. Compare the initial state with the goal state.
5. If both states are equal, display **"Goal State Reached"**.
6. Otherwise, count the number of misplaced tiles.
7. Display the number of misplaced tiles.
8. Print the initial state in 3 × 3 format.
9. Print the goal state in 3 × 3 format.
10. Stop the program.

Flowchart:


START
Define Goal State
[1,2,3,4,5,6,7,8,0]
Read Initial State
from User
Is Initial State
equal to
Goal State?
Yes
No
Print
"Goal State Reached"
Count Number of
Misplaced Tiles
Display Initial State
in 3 × 3 format
Display Goal State
in 3 × 3 format
STOP

Code:

```
# Name: Kola Amrutha Sandhya
# Register Number: 192524270
print("Name            : Kola Amrutha Sandhya")
print("Register Number : 192524270")
goal = [1,2,3,4,9,8,7,6,0]
print("\nEnter the initial state (9 numbers):")
state = list(map(int, input().split()))
if state == goal:
    print("Goal State Reached!")
else:
    count = 0
    for i in range(9):
        if state[i] != goal[i]:
            count += 1
    print("Misplaced Tiles =", count)
print("\nInitial State:")
for i in range(0,9,3):
    print(state[i:i+3])
print("\nGoal State:")
for i in range(0,9,3):
    print(goal[i:i+3])
```

Output:

```
Name            : Kola amrutha Sandhya
Register Number : 192524270

Enter the initial state (9 numbers):
1 2 3 4 9 8 7 6 0
Misplaced Tiles = 3

Initial State:
[1, 2, 3]
[4, 9, 8]
[7, 6, 0]

Goal State:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Result:

The **8-Puzzle Problem** was implemented successfully in Python. The program accepted the initial puzzle state from the user, compared it with the goal state, calculated the number of misplaced tiles, and displayed both the initial and goal states correctly.

2. 8 QUEEN PROBLEM

Aim:

To solve the 8-Queen Problem using Backtracking in Python by placing eight queens on an 8 × 8 chessboard such that no two queens attack each other.

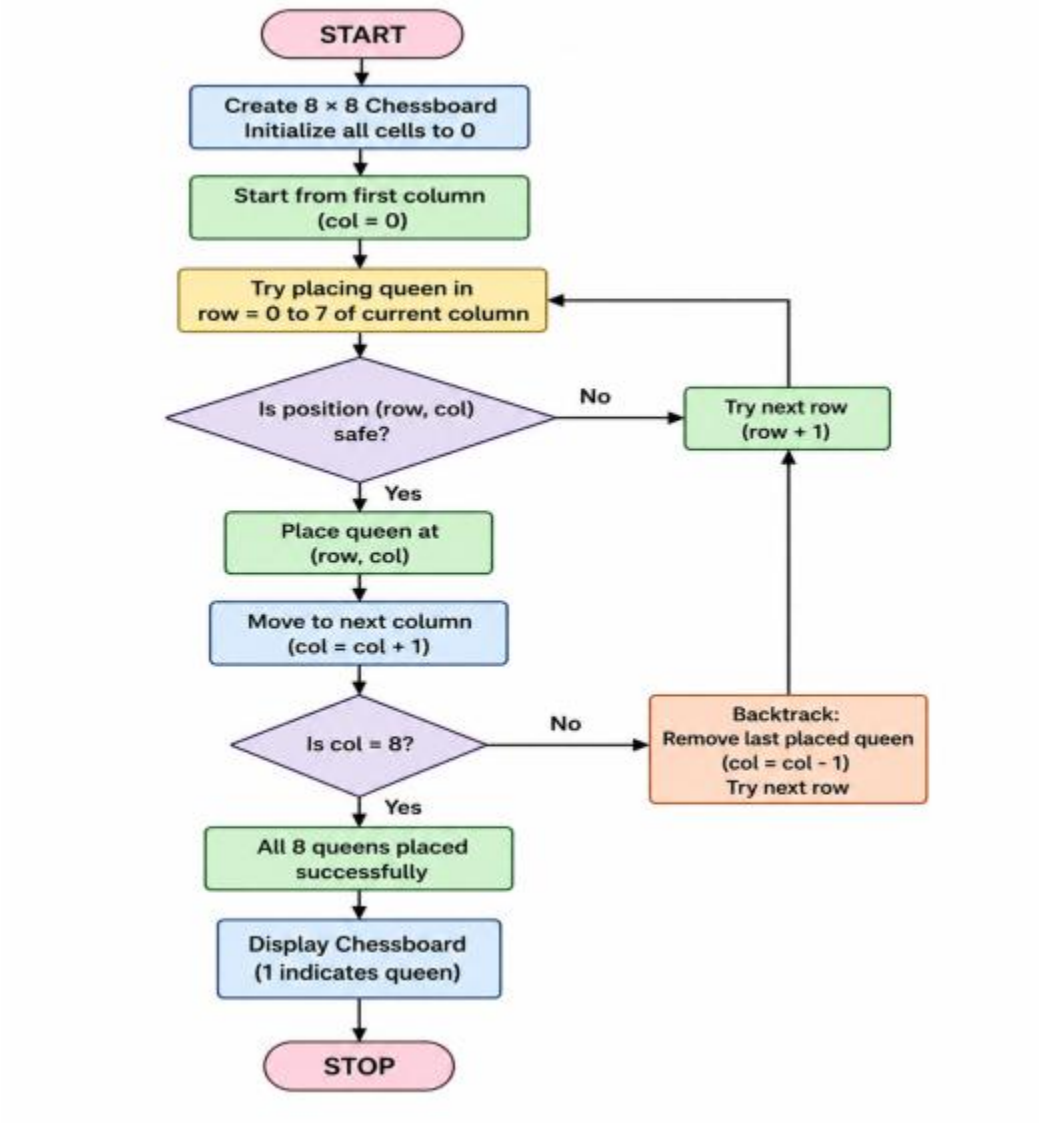
Objective:

- To understand the concept of the 8-Queen Problem.
- To implement the Backtracking algorithm.
- To place eight queens safely on the chessboard.
- To ensure that no two queens share the same row, column, or diagonal.
- To display the valid arrangement of queens.

Algorithm:

1. Start the program.
2. Create an empty 8 × 8 chessboard.
3. Begin placing queens from the first column.
4. Check whether the current position is safe.
5. If safe, place the queen.
6. Move to the next column and repeat the process.
7. If no safe position is found, remove the previously placed queen (Backtracking).
8. Continue until all 8 queens are placed.
9. Display the chessboard.
10. Stop the program.

Flow Chart:


START
Create 8 × 8 Chessboard
Initialize all cells to 0
Start from first column
(col = 0)
Try placing queen in
row = 0 to 7 of current column
Is position (row, col)
safe?
No
Try next row
(row + 1)
Yes
Place queen at
(row, col)
Move to next column
(col = col + 1)
Is col = 8?
No
Backtrack:
Remove last placed queen
(col = col - 1)
Try next row
Yes
All 8 queens placed
successfully
Display Chessboard
(1 indicates queen)
STOP

CODE:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
N = 8
board = [[0 for i in range(N)] for j in range(N)]

def is_safe(board, row, col):
    # Check left side of current row
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper left diagonal
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check lower left diagonal
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
```

```
            return False
        i += 1
        j -= 1

    return True

def solve(board, col):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve(board, col + 1):
                return True

            board[row][col] = 0

    return False

if solve(board, 0):
    print("Solution Found:\n")
    for i in range(N):
        for j in range(N):
            if board[i][j] == 1:
                print("Q", end=" ")
            else:
```

```
            print(".", end=" ")
        print()
else:
    print("No Solution Exists")
```

OUTPUT:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Solution Found:

Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```

Result:

The 8-Queen Problem was successfully solved using the Backtracking algorithm in Python. The program placed eight queens on the chessboard such that no two queens attacked each other and displayed a valid solution.

3. WATER JUG PROBLEM

Aim

To solve the Water Jug Problem using Python by finding the sequence of steps required to obtain the required amount of water.

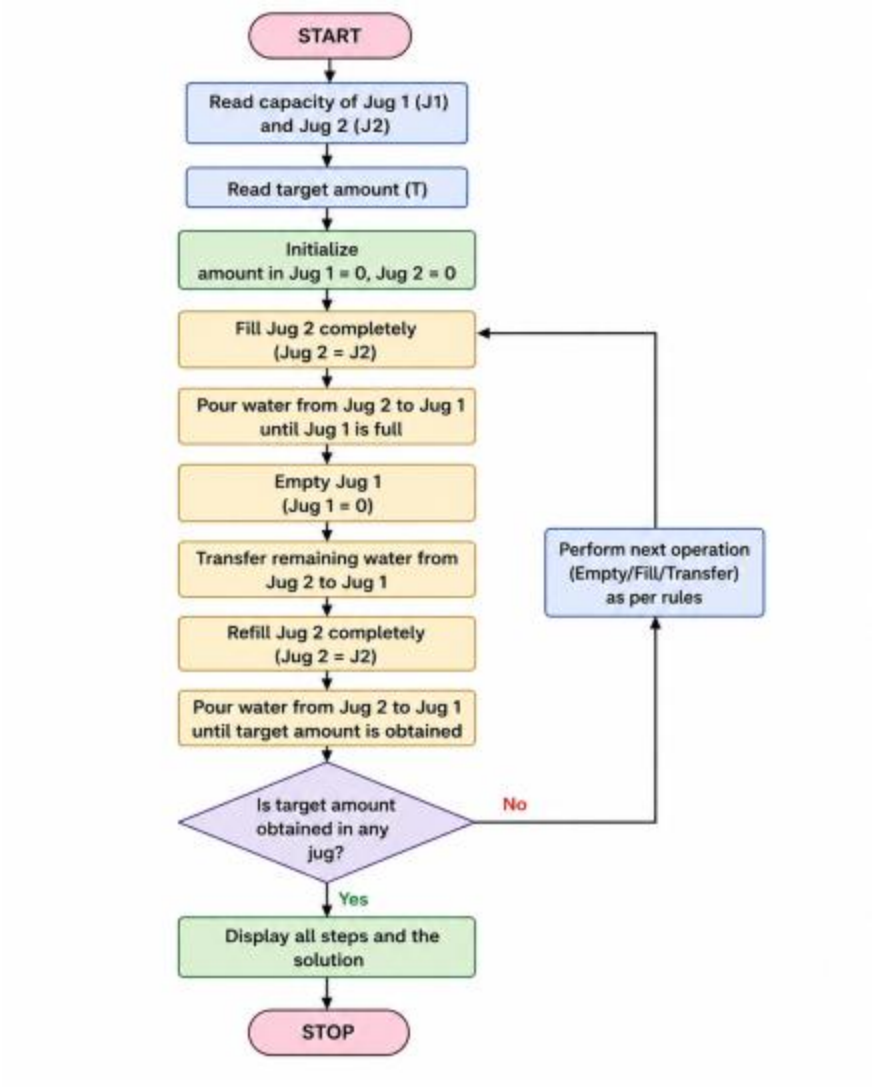
Objective

- To understand the Water Jug Problem.
- To perform fill, empty, and transfer operations.
- To obtain the required quantity of water in one of the jugs.
- To display the sequence of steps leading to the solution.

Algorithm

1. Start the program.
2. Read the capacities of Jug 1 and Jug 2.
3. Read the target amount of water.
4. Fill Jug 2 completely.
5. Pour water from Jug 2 to Jug 1 until Jug 1 is full.
6. Empty Jug 1.
7. Transfer the remaining water from Jug 2 to Jug 1.
8. Refill Jug 2.
9. Pour water from Jug 2 to Jug 1 until the target amount is obtained.
10. Display the steps.
11. Stop the program.

Flow Chart:


START
Read capacity of Jug 1 (J1) and Jug 2 (J2)
Read target amount (T)
Initialize amount in Jug 1 = 0, Jug 2 = 0
Fill Jug 2 completely (Jug 2 = J2)
Pour water from Jug 2 to Jug 1 until Jug 1 is full
Empty Jug 1 (Jug 1 = 0)
Transfer remaining water from Jug 2 to Jug 1
Refill Jug 2 completely (Jug 2 = J2)
Pour water from Jug 2 to Jug 1 until target amount is obtained
Is target amount obtained in any jug?
No
Perform next operation (Empty/Fill/Transfer) as per rules
Yes
Display all steps and the solution
STOP

CODE:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))
a = 0
b = 0
print("\nSteps:")
while b != target:
    if a == 0:
        a = jug1
        print("Fill Jug 1")

    elif b == jug2:
        b = 0
        print("Empty Jug 2")

    else:
        transfer = min(a, jug2 - b)
        a = a - transfer
        b = b + transfer
        print("Transfer water from Jug 1 to Jug 2")
    print("Jug1 =", a, " Jug2 =", b)

print("\nTarget Reached!")
```

OUTPUT:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2

Steps:
Fill Jug 1
Jug1 = 4  Jug2 = 0
Transfer water from Jug 1 to Jug 2
Jug1 = 1  Jug2 = 3
Empty Jug 2
Jug1 = 1  Jug2 = 0
Transfer water from Jug 1 to Jug 2
Jug1 = 0  Jug2 = 1
Fill Jug 1
Jug1 = 4  Jug2 = 1
Transfer water from Jug 1 to Jug 2
Jug1 = 2  Jug2 = 3
Empty Jug 2
Jug1 = 2  Jug2 = 0
Transfer water from Jug 1 to Jug 2
Jug1 = 0  Jug2 = 2

Target Reached!
```

Result:

The Water Jug Problem was implemented successfully in Python. The program accepted the capacities of the two jugs and the target amount as input, performed the fill, empty, and transfer operations, and successfully reached the required amount of water while displaying each step of the solution.

4. VACUUM CLEANER PROBLEM

Aim

To implement the Vacuum Cleaner Problem using Python and simulate a simple intelligent agent that cleans all dirty rooms.

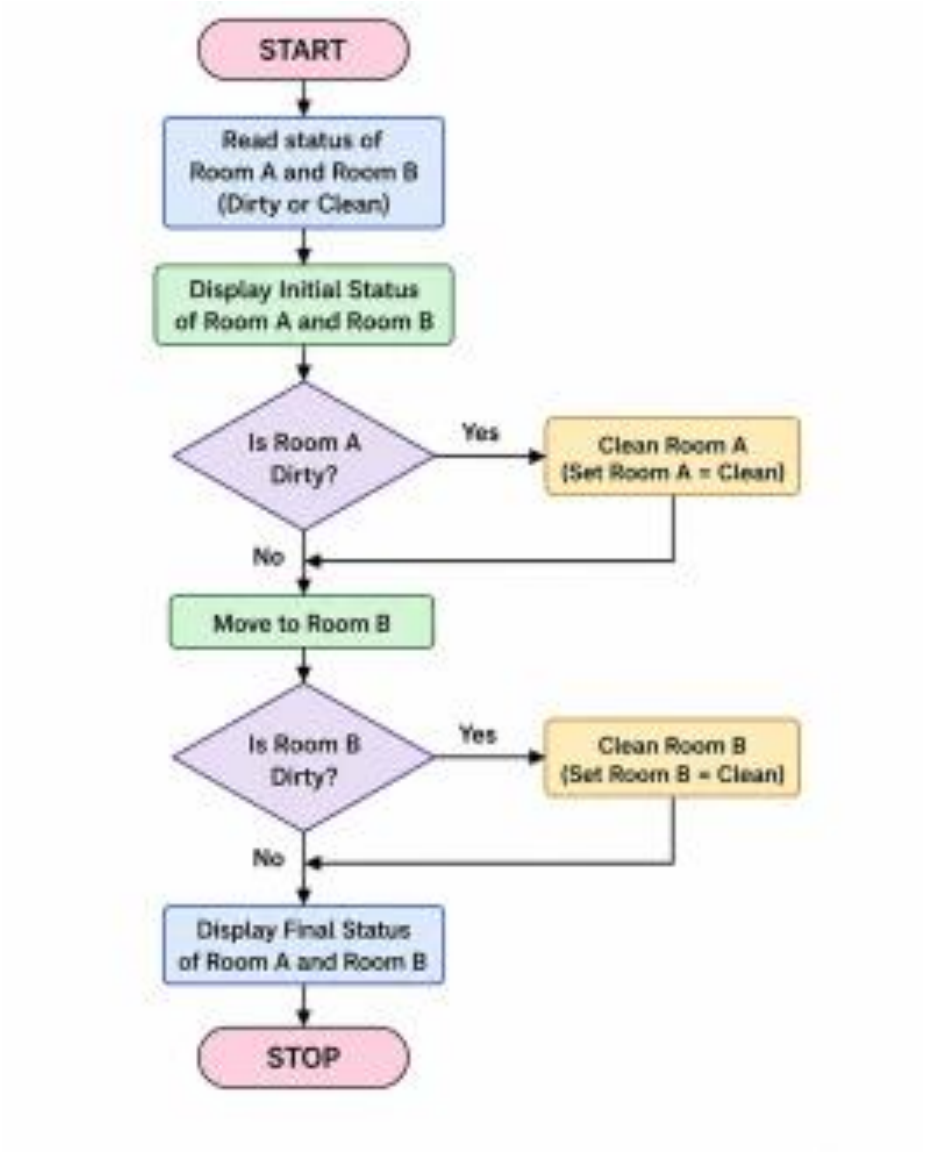
Objective

- To understand the concept of an intelligent agent.
- To simulate the working of a vacuum cleaner agent.
- To clean dirty rooms automatically.
- To display the status of each room before and after cleaning.

Algorithm

1. Start the program.
2. Read the status of Room A and Room B from the user (Dirty or Clean).
3. Check Room A.
4. If Room A is dirty, clean it.
5. Move to Room B.
6. If Room B is dirty, clean it.
7. Display the final status of both rooms.
8. Stop the program.

Flow Chart:


START
Read status of Room A and Room B (Dirty or Clean)
Display Initial Status of Room A and Room B
Is Room A Dirty?
Yes
Clean Room A (Set Room A = Clean)
No
Move to Room B
Is Room B Dirty?
Yes
Clean Room B (Set Room B = Clean)
No
Display Final Status of Room A and Room B
STOP

CODE:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
# Read room status
roomA = input("Enter Room A status (Dirty/Clean): ")
roomB = input("Enter Room B status (Dirty/Clean): ")

print("\nInitial State")
print("Room A =", roomA)
print("Room B =", roomB)

# Clean Room A
if roomA.lower() == "dirty":
    print("\nCleaning Room A...")
    roomA = "Clean"
else:
    print("\nRoom A is already Clean")

# Clean Room B
if roomB.lower() == "dirty":
    print("Cleaning Room B...")
    roomB = "Clean"
else:
    print("Room B is already Clean")

# Display final state
```

```
print("\nFinal State")

print("Room A =", roomA)

print("Room B =", roomB)


print("\nAll rooms are clean.")
```

OUTPUT:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Enter Room A status (Dirty/Clean): Dirty
Enter Room B status (Dirty/Clean): Clean

Initial State
Room A = Dirty
Room B = Clean

Cleaning Room A...
Room B is already Clean

Final State
Room A = Clean
Room B = Clean

All rooms are clean.
```

Result:

The Vacuum Cleaner Problem was implemented successfully in Python. The program accepted the status of two rooms, cleaned the dirty rooms automatically, and displayed the final status, demonstrating the behavior of a simple intelligent agent.

5. BREADTH FIRST SEARCH (BFS)

Aim

To implement the Breadth First Search (BFS) algorithm in Python to traverse all the vertices of a graph level by level.

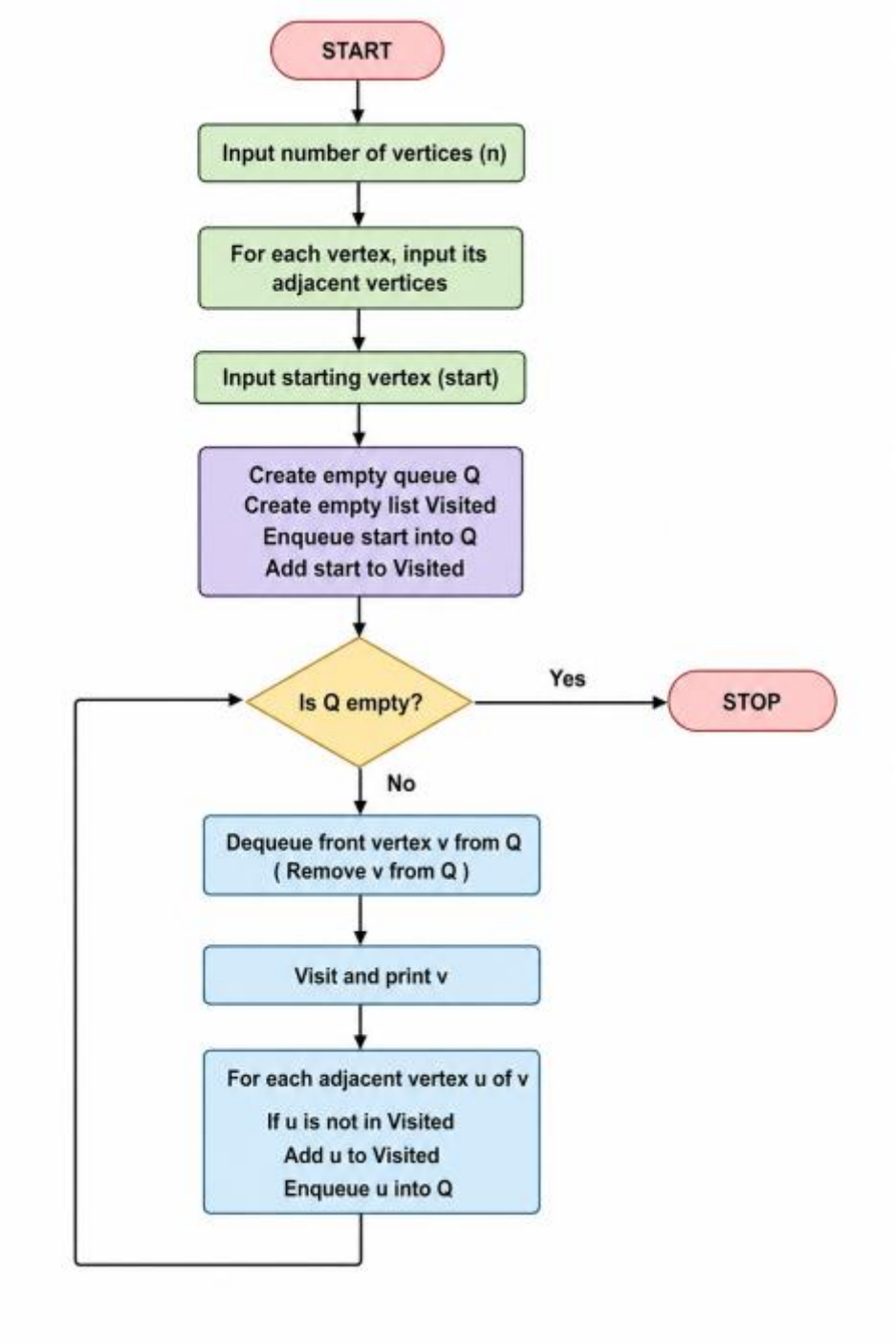
Objective

- To understand the working of the BFS algorithm.
- To traverse all the nodes of a graph starting from a given source node.
- To use a queue for visiting nodes in breadth-wise order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Create an empty list for visited nodes.
4. Create an empty queue.
5. Read the starting node.
6. Add the starting node to the visited list and queue.
7. Repeat until the queue becomes empty:
    - Remove the first node from the queue.
    - Display the node.
    - Visit all its adjacent nodes.
    - If an adjacent node is not visited, add it to the visited list and queue.
8. Stop.

Flow Chart:


START
Input number of vertices (n)
For each vertex, input its adjacent vertices
Input starting vertex (start)
Create empty queue Q
Create empty list Visited
Enqueue start into Q
Add start to Visited
Is Q empty?
Yes
STOP
No
Dequeue front vertex v from Q
(Remove v from Q)
Visit and print v
For each adjacent vertex u of v
If u is not in Visited
Add u to Visited
Enqueue u into Q

CODE:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
graph = {}
visited = []
queue = []

# Read graph
n = int(input("Enter number of vertices: "))

for i in range(n):
    vertex = input("Enter vertex: ")
    neighbors = input("Enter adjacent vertices (space separated): ").split()
    graph[vertex] = neighbors

# BFS Function
def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
```

```
            visited.append(neighbor)
            queue.append(neighbor)

# Read starting vertex
start = input("Enter starting vertex: ")

print("\nBFS Traversal:")
bfs(start)
```

OUTPUT:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Enter number of vertices: 5
Enter vertex: A
Enter adjacent vertices (space separated): B C
Enter vertex: B
Enter adjacent vertices (space separated): D E
Enter vertex: C
Enter adjacent vertices (space separated): E
Enter vertex: D
Enter adjacent vertices (space separated):
Enter vertex: E
Enter adjacent vertices (space separated):
Enter starting vertex: A

BFS Traversal:
A B C D E
```

Result:

The Breadth First Search (BFS) algorithm was successfully implemented in Python. The graph was traversed level by level from the given starting node using a queue, and all reachable nodes were visited successfully.

6.DEPTH FIRST SEARCH (DFS)

Aim

To implement the Depth First Search (DFS) algorithm in Python to traverse all the vertices of a graph using recursion.

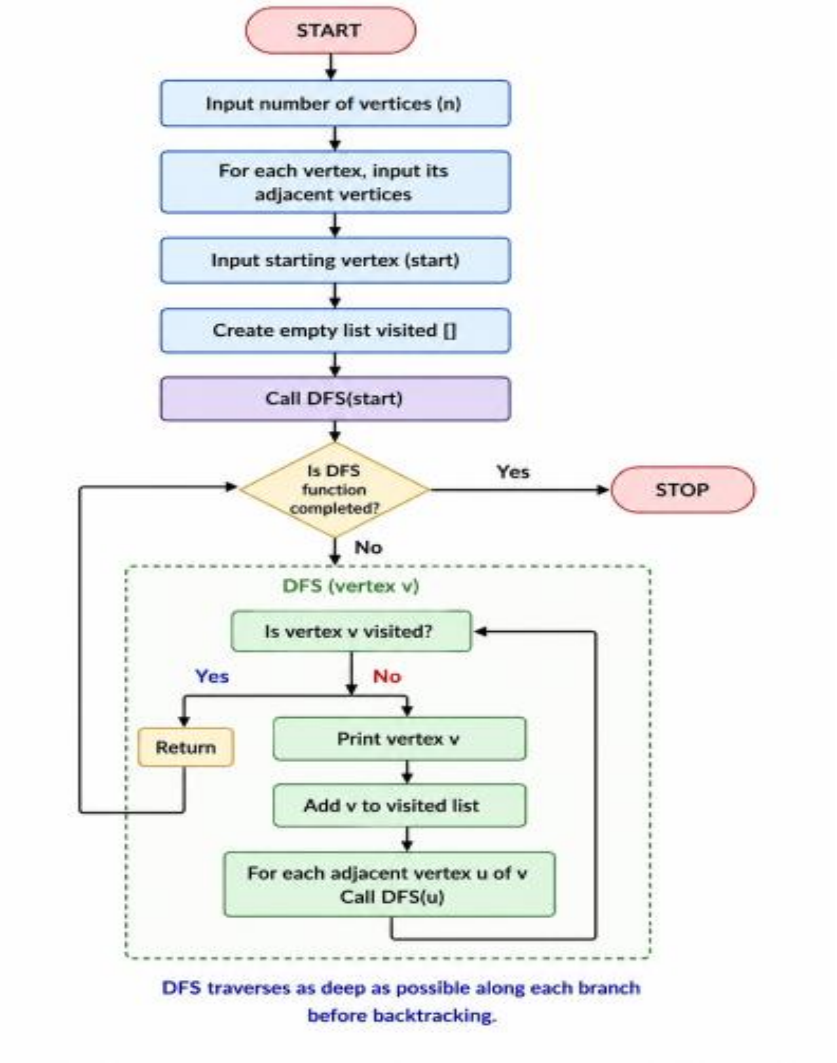
Objective

- To understand the working of the DFS algorithm.
- To traverse all the nodes of a graph from a given starting vertex.
- To use recursion for exploring graph vertices in depth-first order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Create an empty list named visited.
4. Read the number of vertices.
5. Read each vertex and its adjacent vertices.
6. Read the starting vertex.
7. Visit the starting vertex and mark it as visited.
8. Print the current vertex.
9. Recursively visit every unvisited adjacent vertex.
10. Repeat until all reachable vertices are visited.
11. Stop.

Flow Chart:


START
Input number of vertices (n)
For each vertex, input its adjacent vertices
Input starting vertex (start)
Create empty list visited []
Call DFS(start)
Is DFS function completed?
Yes
STOP
No
DFS (vertex v)
Is vertex v visited?
Yes
No
Return
Print vertex v
Add v to visited list
For each adjacent vertex u of v
Call DFS(u)
DFS traverses as deep as possible along each branch before backtracking.

Code:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
visited = []
graph = {}

n = int(input("Enter number of vertices: "))

for i in range(n):
   vertex = input("Enter vertex: ")
neighbors = input("Enter adjacent vertices (space separated): ").split()
   graph[vertex] = neighbors

def dfs(node):
   if node not in visited:
print(node, end=" ")
visited.append(node)

      for i in graph[node]:
dfs(i)

start = input("\nEnter starting vertex: ")

print("\nDFS Traversal:")
dfs(start)
```

Output:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Enter number of vertices: 5
Enter vertex: A
Enter adjacent vertices (space separated): B C
Enter vertex: B
Enter adjacent vertices (space separated): D E
Enter vertex: C
Enter adjacent vertices (space separated): E
Enter vertex: D
Enter adjacent vertices (space separated):
Enter vertex: E
Enter adjacent vertices (space separated):
Enter starting vertex: A

DFS Traversal:
A B D E C
```

Result:

The Depth First Search (DFS) algorithm was successfully implemented in Python. The graph was traversed by visiting each vertex recursively in depth-first order, and all reachable vertices were visited exactly once.

7. MISSIONARIES AND CANNIBALS PROBLEM

Aim

To solve the Missionaries and Cannibals problem using Python by safely transporting all missionaries and cannibals across the river without ever allowing cannibals to outnumber missionaries on either bank.

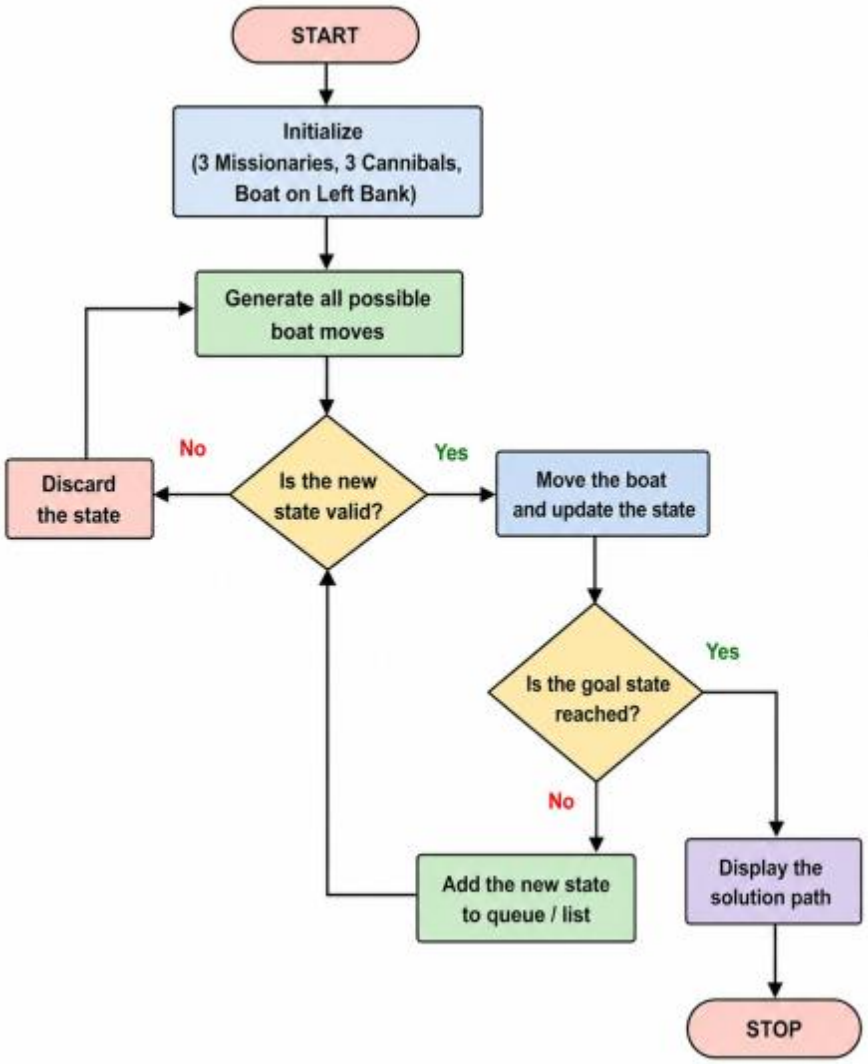
Objective

- To understand the Missionaries and Cannibals problem.
- To implement a Python solution.
- To ensure all safety constraints are satisfied.
- To find a valid sequence of moves from the initial state to the goal state.

Algorithm

1. Start.
2. Represent the initial state as (3 Missionaries, 3 Cannibals, Left Bank).
3. Represent the goal state as (0 Missionaries, 0 Cannibals, Right Bank).
4. Define all possible boat moves.
5. Check whether each generated state is valid.
6. Move the boat between banks.
7. Continue exploring valid states until the goal is reached.
8. Display the solution path.
9. Stop

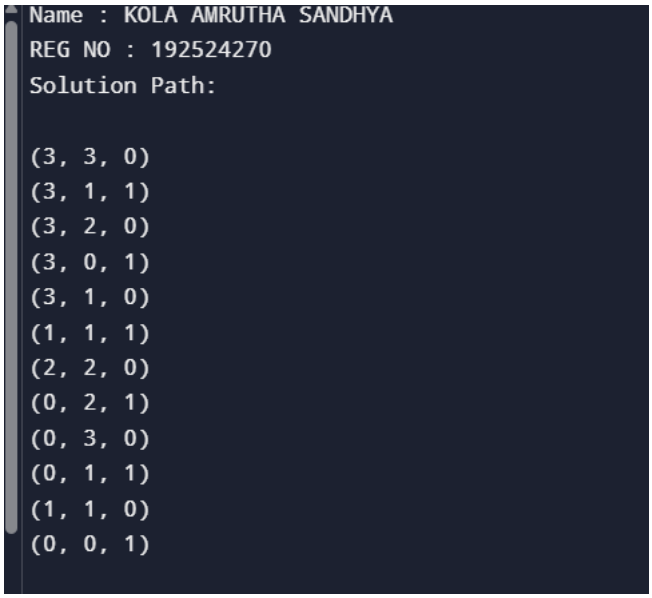
Flow Chart:


START
Initialize
(3 Missionaries, 3 Cannibals,
Boat on Left Bank)
Generate all possible
boat moves
Is the new
state valid?
No
Yes
Discard
the state
Move the boat
and update the state
Is the goal state
reached?
Yes
No
Add the new state
to queue / list
Display the
solution path
STOP

Code:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
from collections import deque

q = deque([((3,3,0), [])])
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
seen = set()

while q:
    (m,c,b), path = q.popleft()
    if (m,c,b) in seen:
        continue
seen.add((m,c,b))

    if (m,c,b) == (0,0,1):
print(path + [(m,c,b)])
        break

    for x,y in moves:
        nm = m-x if b==0 else m+x
        nc = c-y if b==0 else c+y
        if 0<=nm<=3 and 0<=nc<=3:
            if (nm==0 or nm>=nc) and (3-nm==0 or 3-nm>=3-nc):
q.append(((nm,nc,1-b), path+[(m,c,b)]))
```

Output:



Result:

The Missionaries and Cannibals problem was successfully solved using Python. The program found a valid sequence of moves that safely transported all three missionaries and three cannibals across the river while satisfying all safety constraints.

8. TRAVELLING SALESMAN PROBLEM

Aim

To solve the Travelling Salesman Problem using Python by finding the shortest route that visits each city exactly once and returns to the starting city.

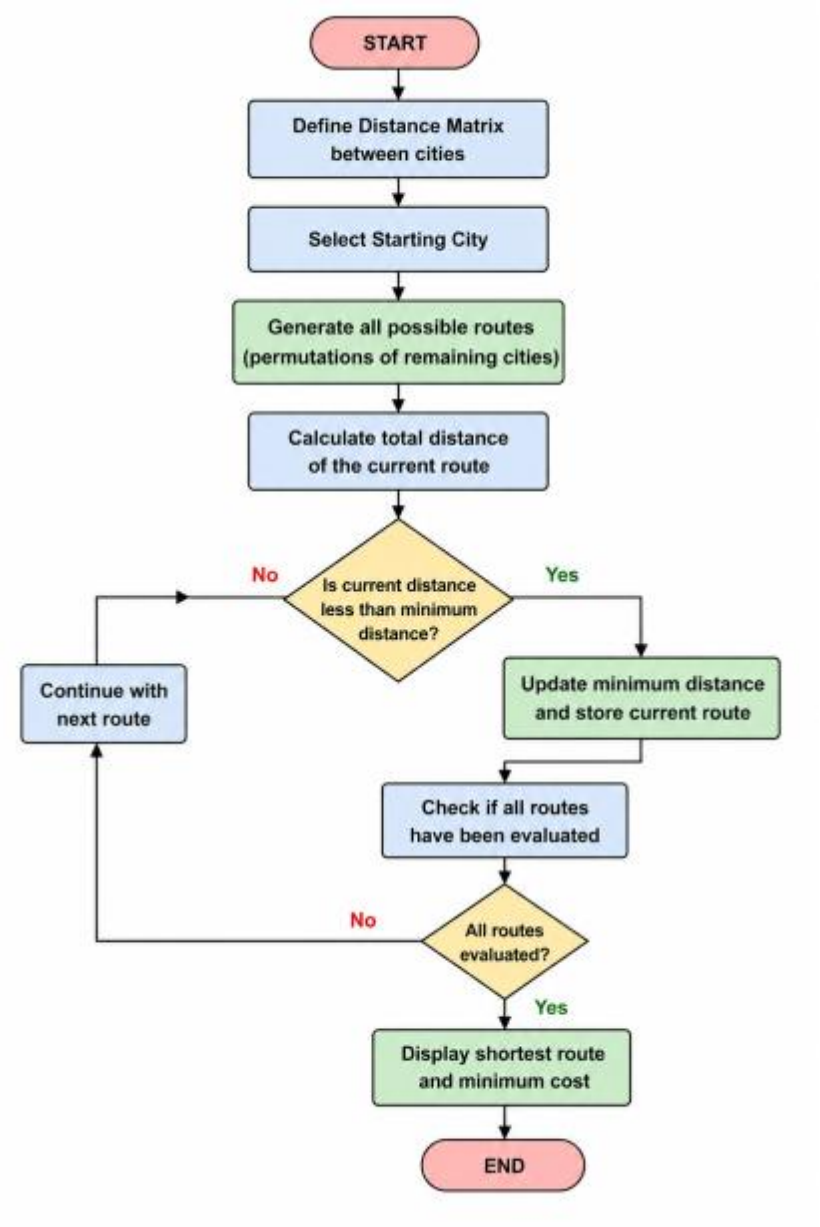
Objective

- To understand the Travelling Salesman Problem (TSP).
- To find the minimum-cost tour among all possible routes.
- To implement a simple Python program for TSP.
- To display the shortest path and its total cost.

Algorithm

1. Start.
2. Define the distance matrix between cities.
3. Fix the starting city.
4. Generate all possible routes for the remaining cities.
5. Calculate the total distance of each route.
6. Compare all route distances.
7. Select the route with the minimum distance.
8. Display the shortest route and its cost.
9. Stop.

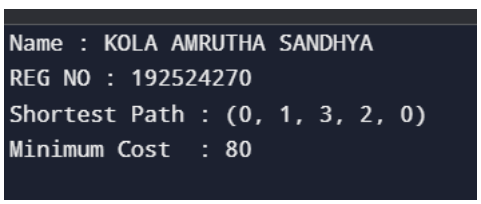
Flow Chart:


START
Define Distance Matrix between cities
Select Starting City
Generate all possible routes (permutations of remaining cities)
Calculate total distance of the current route
Is current distance less than minimum distance?
No
Yes
Continue with next route
Update minimum distance and store current route
Check if all routes have been evaluated
All routes evaluated?
No
Yes
Display shortest route and minimum cost
END

Code:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
from itertools import permutations

d = [[0,10,15,20],
     [10,0,35,25],
     [15,35,0,30],
     [20,25,30,0]]

cost = 999
path = ()

for p in permutations([1,2,3]):
    c = d[0][p[0]] + d[p[0]][p[1]] + d[p[1]][p[2]] + d[p[2]][0]
    if c < cost:
        cost = c
        path = (0,) + p + (0,)

print("Shortest Path:", path)
print("Minimum Cost:", cost)
```

Output:



Result:

The Travelling Salesman Problem was successfully solved using Python. The program generated all possible routes, compared their travel costs, and identified the shortest route with the minimum total distance.

9. CRYPT ARITHMETIC PROBLEM

Aim

To solve a Crypt Arithmetic problem using Python by assigning unique digits to letters such that the arithmetic equation is satisfied.

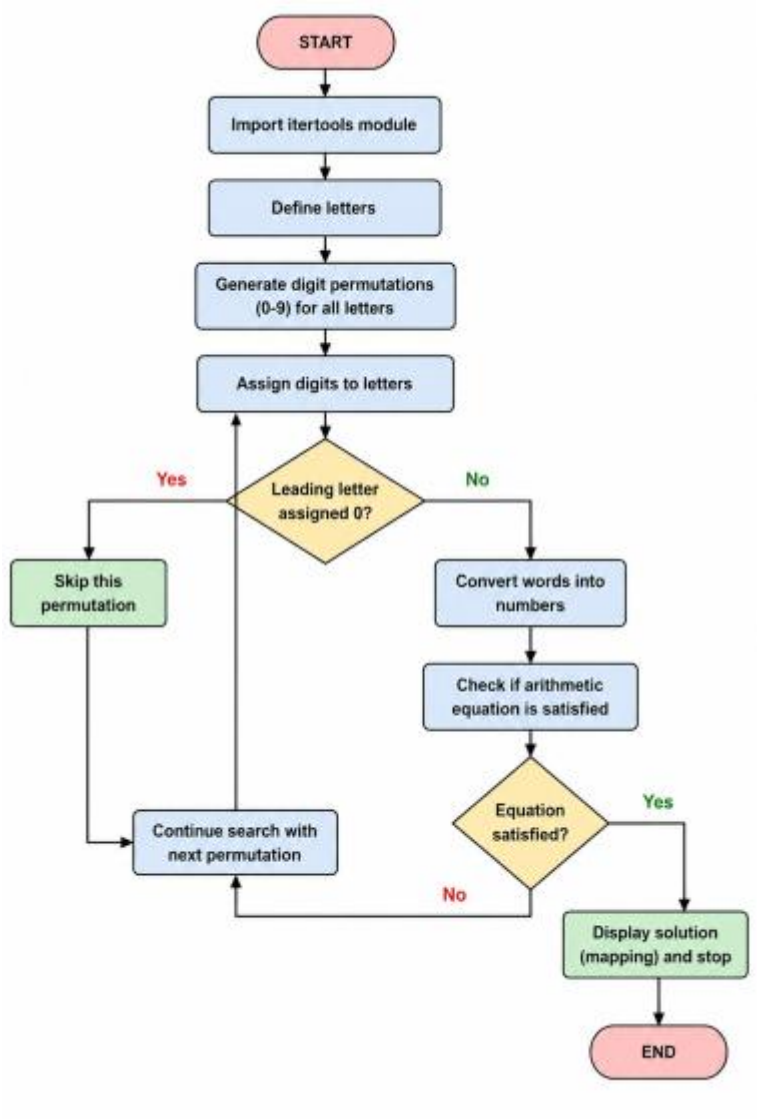
Objective

- To understand the concept of Crypt Arithmetic problems.
- To use Python to generate valid digit assignments.
- To ensure each letter is assigned a unique digit.
- To display the correct solution for the given puzzle.

Algorithm

1. Start.
2. Import the itertools module.
3. Define the letters used in the crypt arithmetic puzzle.
4. Generate all possible unique digit assignments.
5. Assign digits to each letter.
6. Ensure leading letters are not assigned zero.
7. Convert the words into numbers.
8. Check whether the arithmetic equation is satisfied.
9. If a valid solution is found, display the mapping and stop.
10. End.

Flow Chart


START
Import itertools module
Define letters
Generate digit permutations (0-9) for all letters
Assign digits to letters
Leading letter assigned 0?
Yes
No
Skip this permutation
Convert words into numbers
Check if arithmetic equation is satisfied
Equation satisfied?
Yes
No
Continue search with next permutation
Display solution (mapping) and stop
END

Code:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
from itertools import permutations

for p in permutations(range(10), 8):
S,E,N,D,M,O,R,Y = p
   if S and M:
      send = 1000*S+100*E+10*N+D
      more = 1000*M+100*O+10*R+E
      money = 10000*M+1000*O+100*N+10*E+Y
      if send + more == money:
print("SEND =", send)
print("MORE =", more)
print("MONEY =", money)
         break
```

Output:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Solution Found!

SEND  = 9567
MORE  = 1085
MONEY = 10652

Letter Values
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
```

Result:

The Crypt Arithmetic problem was successfully solved using Python. The program assigned unique digits to each letter and verified the equation SEND + MORE = MONEY, producing the correct solution.

10. A* ALGORITHM

Aim

To implement the A* (A-Star) Search Algorithm in Python to find the shortest path from a start node to a goal node using heuristic values.

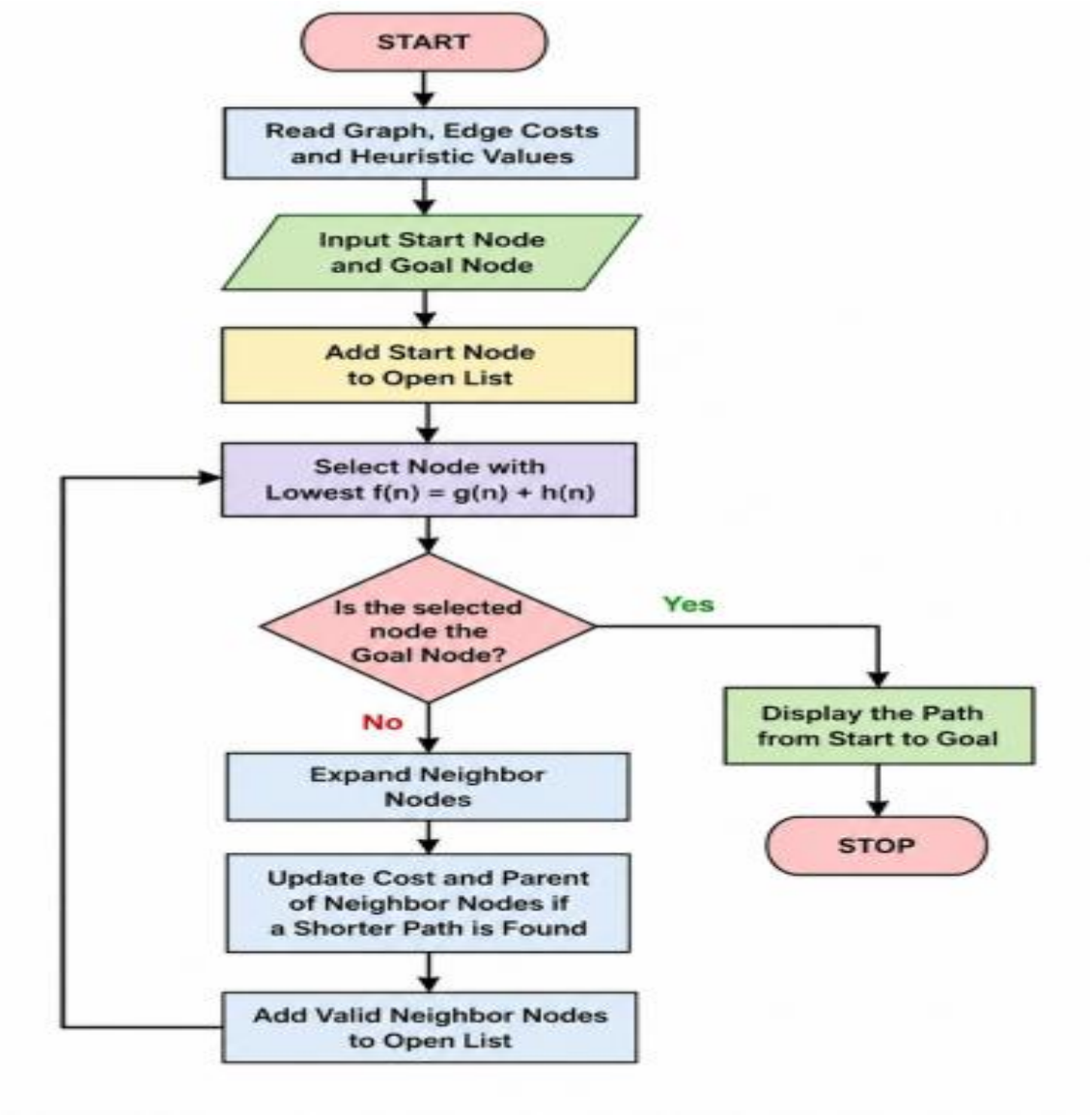
Objective

- To understand the working of the A* search algorithm.
- To find the optimal path between the start and goal nodes.
- To use heuristic values for efficient path searching.
- To implement the algorithm using Python with manual input.

Algorithm

1. Start the program.
2. Read the graph, edge costs, and heuristic values from the user.
3. Enter the start node and goal node.
4. Add the start node to the open list.
5. Select the node with the minimum **f(n) = g(n) + h(n)** value.
6. If the selected node is the goal node, display the path and stop.
7. Otherwise, expand its neighboring nodes.
8. Update the cost and parent of each neighboring node if a shorter path is found.
9. Repeat Steps 5–8 until the goal is reached or the open list becomes empty.
10. Stop the program.

Flowchart:


START
Read Graph, Edge Costs and Heuristic Values
Input Start Node and Goal Node
Add Start Node to Open List
Select Node with Lowest f(n) = g(n) + h(n)
Is the selected node the Goal Node?
Yes
No
Display the Path from Start to Goal
STOP
Expand Neighbor Nodes
Update Cost and Parent of Neighbor Nodes if a Shorter Path is Found
Add Valid Neighbor Nodes to Open List

Python Code:

```
# Name: KOLA AMRUTHA SANDHYA
# REG NO : 192524270
print("Name : KOLA AMRUTHA SANDHYA")
print("REG NO : 192524270")
graph = {}
n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Node: ")
    graph[node] = {}
    e = int(input("Edges from " + node + ": "))
    for j in range(e):
        v = input("Connected node: ")
        c = int(input("Cost: "))
        graph[node][v] = c

h = {}
for i in graph:
    h[i] = int(input("Heuristic of " + i + ": "))

start = input("Start node: ")
goal = input("Goal node: ")

open = [start]
cost = {start: 0}
parent = {start: None}
```

```
while open:
    x = min(open, key=lambda i: cost[i] + h[i])
open.remove(x)


    if x == goal:
        path = []
        while x:
path.append(x)
            x = parent[x]
print("Path:", path[::-1])
        break


    for y in graph[x]:
        g = cost[x] + graph[x][y]
        if y not in cost or g < cost[y]:
            cost[y] = g
            parent[y] = x
open.append(y)
```

Output:

```
Name : KOLA AMRUTHA SANDHYA
REG NO : 192524270
Enter number of nodes: 4
Enter node: A
Enter number of edges from A: 2
Enter connected node: B
Enter cost: 1
Enter connected node: C
Enter cost: 1
Enter node: B
Enter number of edges from B: 1
Enter connected node: C
Enter cost: 5
Enter node: C
Enter number of edges from C: 1
Enter connected node: D
Enter cost: 3
Enter node: D
Enter number of edges from D: 1
Enter connected node: A
Enter cost: 5

Enter Heuristic Values
Heuristic of A: 7
Heuristic of B: 6
Heuristic of C: 3
Heuristic of D: 2

Enter Start Node: A
Enter Goal Node: D

Shortest Path: A -> C -> D
Total Cost: 4
```

Result:

The A* Search Algorithm was successfully implemented in Python. The algorithm efficiently found the shortest path from the start node to the goal node using the heuristic values and path costs.